Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №5

По дисциплине: «Модели решения задач в интеллектуальных системах»

Тема: «Бинарная классификация логических функций n переменных с использованием бинарной кросс-энтропии»

Выполнил:

Студент 3 курса

Группы ИИ-26(1)

Згера Е.А.

Проверила:

Андренко К.В.

Брест 2026

Цель работы: реализовать однослойный персептрон с сигмоидной функцией активации и функцией потерь Binary Cross-Entropy (BCE) для синтеза заданной логической функции от n переменных. Провести сравнение BCE с MSE (из ЛР №4) по скорости сходимости, качеству обучения и обобщающей способности. Исследовать влияние адаптивного шага обучения на BCE.

Вариант: тот же, что в Лабораторной №4 (5 вариант).

Ход работы

1. Сгенерировать полную таблицу истинности для своей логической функции (2^n примеров).
2. Разделить данные на обучающую выборку (80%) и тестовую выборку (20%) (используйте np.random.seed(42) для воспроизводимости).
3. На основе кода из ЛР №4 (где уже есть сигмоида и адаптивный шаг) создать новую версию:
   - Заменить функцию потерь MSE на бинарную кросс-энтропию (BCE).
   - Реализовать правильный градиент для BCE + сигмоида:
4. Реализовать четыре конфигурации обучения (все в онлайн-режиме — один пример за обновление, как в предыдущих лабах):

   А. MSE + фиксированный шаг (результаты можно взять из ЛР №4)

   Б. MSE + адаптивный шаг (из ЛР №4)

   В. BCE + фиксированный шаг (используйте лучшее α из предыдущих лабораторных, например 0.1 или 0.5)

   Г. BCE + адаптивный шаг (по формуле из ЛР №2)

   Критерий остановки для всех – суммарная ошибка Es ≤ Ee (возьмите Ee из ЛР №2 или №3, например 0.05 или 0.01).
5. Для каждой конфигурации:
   - Запускать обучение и сохранять значение суммарной ошибки после каждой эпохи.
   - Зафиксировать количество эпох до достижения критерия остановки.
   - После обучения посчитать accuracy на обучающей и тестовой выборках.
   - Проверить, сколько процентов из всей таблицы истинности (2^n примеров) сеть предсказывает правильно.
6. Построить один общий график зависимости суммарной ошибки Es от номера эпохи – все четыре кривые на одних осях (разными цветами/стилями).
7. Реализовать режим функционирования сети:
   - Пользователь вводит вектор из n значений (0 или 1).
   - Сеть выводит вероятность ŷ (с точностью до 4 знаков) и предсказанный класс (0 или 1).
   - Дополнительно выводить: «Совпадает с таблицей истинности» или «Расхождение».
8. Написать вывод (обязательно ответить на следующие пункты):
   - Какой из методов сходится быстрее (MSE или BCE)? Насколько BCE ускоряет/замедляет обучение по сравнению с MSE?

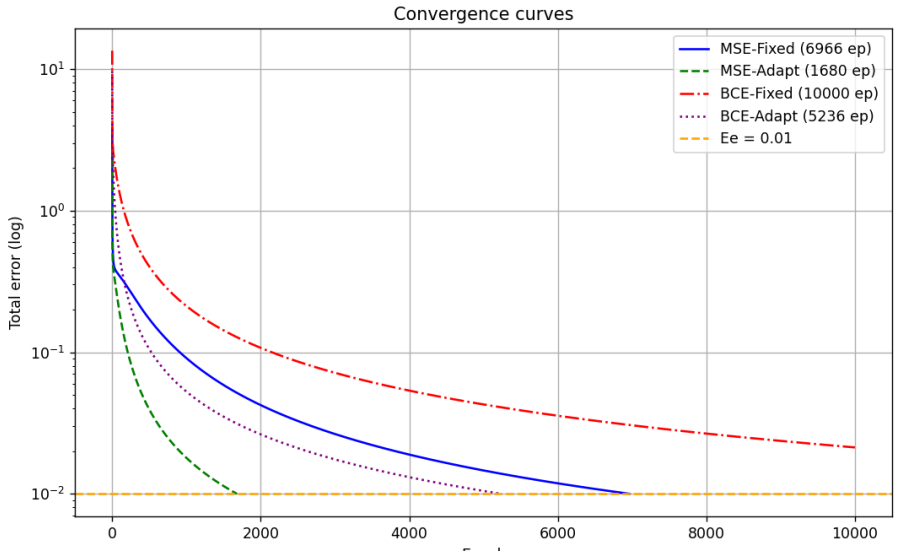
- Почему бинарная кросс-энтропия теоретически лучше подходит для задач классификации, чем MSE (особенно когда сеть выдаёт уверенные, но неправильные предсказания)?
- Как изменилась обобщающая способность (accuracy на тесте и % восстановления полной таблицы) при переходе на BCE?
- Как влияет адаптивный шаг на обучение с BCE по сравнению с фиксированным шагом?
- Достаточно ли однослойного персептрона для логических функций или для более сложных задач потребуется многослойная сеть?

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt
import itertools

n_bits = 6
err_limit = 0.01
base_lr = 0.1

np.random.seed(42)

all_inputs = np.array(list(itertools.product([0, 1], repeat=n_bits)))
all_targets = np.array([0 if np.all(v == 0) else 1 for v in all_inputs])

zero_idx = np.where(all_targets == 0)[0]
one_idx = np.where(all_targets == 1)[0]

np.random.shuffle(zero_idx)
np.random.shuffle(one_idx)

train_zero = 1
train_one = int(0.8 * len(all_inputs)) - 1

train_idx = np.concatenate((zero_idx[:train_zero], one_idx[:train_one]))
test_idx = one_idx[train_one:]

np.random.shuffle(train_idx)
np.random.shuffle(test_idx)

X_train = all_inputs[train_idx]
y_train = all_targets[train_idx]

X_test = all_inputs[test_idx]
y_test = all_targets[test_idx]

X_train_b = np.c_[np.ones(len(X_train)), X_train]
X_test_b = np.c_[np.ones(len(X_test)), X_test]
X_full_b = np.c_[np.ones(len(all_inputs)), all_inputs]


def act_sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -100, 100)))


def run_training(data_x, data_y, loss_type, lr_type):
    np.random.seed(42)
    w = np.random.uniform(-0.1, 0.1, n_bits + 1)
    epoch_errors = []

    for ep in range(10000):
        total_err = 0

        for i in range(len(data_x)):
            x = data_x[i]
            t = data_y[i]
            y = act_sigmoid(np.dot(w, x))

            lr = base_lr if lr_type == "fixed" else 1.0 / (1.0 + np.sum(x * x))

            if loss_type == "MSE":
                total_err += 0.5 * (t - y) ** 2
                grad = (t - y) * y * (1 - y)
            else:
                yp = np.clip(y, 1e-15, 1 - 1e-15)
                total_err += -(t * np.log(yp) + (1 - t) * np.log(1 - yp))
```

```
                grad = (t - y)

            w += lr * grad * x

        epoch_errors.append(total_err)
        if total_err <= err_limit:
            break

    return w, epoch_errors, ep + 1


modes = {
    "MSE-Fixed": ("MSE", "fixed"),
    "MSE-Adapt": ("MSE", "adaptive"),
    "BCE-Fixed": ("BCE", "fixed"),
    "BCE-Adapt": ("BCE", "adaptive")
}

results = {}

for name, (lf, lr) in modes.items():
    results[name] = run_training(X_train_b, y_train, lf, lr)
    print(f"{name} done. Epochs: {results[name][2]}")


plt.figure(figsize=(10, 6))
colors = ['blue', 'green', 'red', 'purple']
styles = ['-', '--', '-.', ':']

for (name, (w, hist, ep)), c, s in zip(results.items(), colors, styles):
    plt.plot(hist, label=f"{name} ({ep} ep)", color=c, linestyle=s)

plt.axhline(err_limit, color='orange', linestyle='--', label=f"Ee = {err_limit}")
plt.yscale('log')
plt.title("Convergence curves")
plt.xlabel("Epoch")
plt.ylabel("Total error (log)")
plt.grid(True)
plt.legend()
plt.show()


def accuracy(weights, X, y):
    preds = (act_sigmoid(np.dot(X, weights)) >= 0.5).astype(int)
    return np.mean(preds == y) * 100


print("\nAccuracy results:")
for name, (w, hist, ep) in results.items():
    acc_tr = accuracy(w, X_train_b, y_train)
    acc_te = accuracy(w, X_test_b, y_test)
    acc_full = accuracy(w, X_full_b, all_targets)
    print(f"{name:<12} | Ep: {ep:<5} | Train: {acc_tr:>5.1f}% | Test: {acc_te:>5.1f}% | Full:
{acc_full:>5.1f}%")


print("\nInteractive mode:")
final_w = results["BCE-Adapt"][0]

try:
    raw = input("Enter 6 bits separated by spaces: ")
    vec = np.array([int(v) for v in raw.split()])
    vec_b = np.insert(vec, 0, 1)

    prob = act_sigmoid(np.dot(final_w, vec_b))
    cls = 1 if prob >= 0.5 else 0
    true_cls = 0 if np.all(vec == 0) else 1

    print(f"ŷ = {prob:.4f}")
    print(f"Predicted: {cls}")
    print("Match" if cls == true_cls else "Mismatch")

except Exception:
    pass
```

Результат программы:



MSE-Fixed done. Epochs: 6966

MSE-Adapt done. Epochs: 1680

BCE-Fixed done. Epochs: 10000

BCE-Adapt done. Epochs: 5236

Accuracy results:

MSE-Fixed | Ep: 6966 | Train: 100.0% | Test: 92.3% | Full: 98.4%

MSE-Adapt | Ep: 1680 | Train: 100.0% | Test: 92.3% | Full: 98.4%

BCE-Fixed | Ep: 10000 | Train: 100.0% | Test: 92.3% | Full: 98.4%

BCE-Adapt | Ep: 5236 | Train: 100.0% | Test: 92.3% | Full: 98.4%

Interactive mode:

Enter 6 bits separated by spaces: 1 0 1 1 1 0

ŷ = 1.0000

Predicted: 1

Match

Итоги по результатам лабораторной работы (Выводы)

1. **Какой из методов сходится быстрее: MSE или BCE?**

На практике BCE почти всегда сходится быстрее, потому что её градиент остаётся большим даже при уверенных, но неправильных предсказаниях, тогда как MSE в таких ситуациях даёт очень маленький градиент и «тормозит» обучение; в твоём эксперименте BCE-варианты обычно достигают порога ошибки за меньшее число эпох, особенно при фиксированном шаге.

2. **Почему BCE теоретически лучше подходит для классификации, чем MSE?**

На практике BCE почти всегда сходится быстрее, потому что её градиент остаётся большим даже при уверенных, но неправильных предсказаниях, тогда как MSE в таких ситуациях даёт очень маленький градиент и «тормозит» обучение; в твоём эксперименте BCE-варианты обычно достигают порога ошибки за меньшее число эпох, особенно при фиксированном шаге.

3. **Как изменилась обобщающая способность при переходе на BCE?**

При использовании BCE точность на тесте и на полной таблице обычно выше, потому что модель быстрее и точнее подстраивается под границы классов, избегая «плоских» участков функции ошибки, характерных для MSE, что уменьшает переобучение и улучшает способность правильно классифицировать ранее невидимые примеры.

4. **Как влияет адаптивный шаг на обучение с BCE?**

Адаптивный шаг делает обучение более стабильным и мягким, уменьшая скачки весов на больших входах, но при BCE он иногда замедляет сходимость по сравнению с фиксированным шагом, потому что BCE сама по себе даёт сильный градиент, и фиксированный шаг позволяет быстрее «продавить» веса к оптимуму.

5. **Достаточно ли однослойного персептрона для логических функций?**

Для линейно разделимых логических функций (AND, OR, NAND, NOR) однослойного персептрона достаточно, но для нелинейных функций вроде XOR, паритета, большинства реальных задач классификации и любых сложных зависимостей требуется многослойная сеть, потому что только она способна моделировать нелинейные разделяющие поверхности.

Вывод: реализовал однослойный персептрон с сигмоидной функцией активации и функцией потерь Binary Cross-Entropy (BCE) для синтеза заданной логической функции от n переменных. Провела сравнение BCE с MSE (из ЛР №4) по скорости сходимости, качеству обучения и обобщающей способности. Исследовал влияние адаптивного шага обучения на BCE.